

C++ Standard Proposal — A Time-Zone Library

Document number:	P0216R0
Date:	2016-02-11
Reply-to:	Greg Miller (igm@google.com), Bradley White (bww@google.com)
Audience:	Library Evolution Working Group

Table of Contents

- [Introduction](#)
- [Motivation and Scope](#)
- [The Conceptual Model and Terminology](#)
- [Impact on the Standard](#)
- [Design Decisions](#)
- [Technical Specification](#)
- [Synopsis](#)
- [Examples](#)
- [References](#)

Introduction

Note: This proposal depends on the Civil-Time Library that is proposed in [P0215R0](#).

This document proposes a standard C++ library for computing with real-world time zones, such as "America/New_York", "Europe/London", and "Australia/Sydney". The data describing the rules of each time zone are distributed separately (e.g., <https://www.iana.org/time-zones>), and are not a part of this proposal. The core of this paper proposes a single user-facing class that can represent any real-world time zone and is able to convert between the *absolute-time* and the *civil-time* domains (described below). All the complex time-zone computations will be handled by this Time-Zone Library. The exposed interface will give callers access to all the time-zone information they need, while encouraging proper time-programming practices.

Motivation and Scope

Programming with time zones is notoriously difficult and error prone. There is no simple conceptual model to help programmers reason about this problem space, and there is very little library support. The existing C and C++ standards provide limited support for the "UTC" and "local" time zones, but there is minimal support for other arbitrary time zones.

Sadly, programmers must each become time-zone "experts" in order to accomplish their goals. In code, it is not uncommon to see the addition/subtraction of a UTC offset and a time point, preceded by volumes of commenting to explain why that operation is necessary, usually with some caveat about daylight-saving time (DST). An informal survey of such call sites in Google code showed that few of the operations were actually correct, and nearly all of the comments were misinformed. This is likely because it is impractical to expect time-zone expertise from all programmers.

Programmers should not need to do their own arithmetic with time-zone offsets. This is a frequent source of bugs and is an anti-pattern we call "epoch shifting" (CppCon 2015 talk: <https://youtu.be/2rnIHsqABfM?t=12m30s>). A proper time-zone library must do all necessary time-zone arithmetic itself, giving callers higher-level abstractions on which to build their programs. These higher-level abstractions must form an effective conceptual model about how time zones work so programmers can correctly reason about their code.

The Time-Zone Library proposed here has been implemented and widely used within Google for the past few years. It is actively being used daily, in real-world code, by novice and expert programmers alike.

The Conceptual Model and Terminology

The concept of time zones, as described in this proposal, is just one of three concepts that comprise a complete, straightforward, and language-neutral model for reasoning about time programming: *absolute time*, *civil time*, and *time zone*.

Absolute time uniquely and universally represents a specific instant in time. It has no notion of calendars, or dates, or times of day. Instead, it is a measure of the passage of real time, typically as a simple count of ticks since some epoch. Absolute times are independent of all time zones and do not suffer from human-imposed complexities such as daylight-saving time (DST). Many C++ types exist to represent absolute times, classically `time_t` and more recently `std::chrono::time_point`.

Civil time is the legally recognized representation of time for ordinary affairs (cf. <http://www.merriam-webster.com/dictionary/civil>). It is a human-scale representation of time that consists of the six fields — year, month, day, hour, minute, and second (sometimes shortened to "YMDHMS") — and it follows the rules of the [Proleptic Gregorian Calendar](#), with 24-hour days divided into 60-minute hours and 60-second minutes. Like absolute times, civil times are also independent of all time zones and their related complexities (e.g., DST). While `std::tm` contains the six civil-time fields (YMDHMS), plus a few more, it does not have behavior to enforce the rules of civil time.

Time zones are geo-political regions within which human-defined rules are shared to convert between absolute-time and civil-time domains. A time zone's rules include things like the region's offset from the [UTC](#) time standard, daylight-saving adjustments, and short abbreviation strings. Time zones often have a history of disparate rules that apply only for certain periods, because the rules may change at the whim of a region's local government. For this reason, time-zone rules are usually compiled into data snapshots that are used at runtime to perform conversions between absolute and civil times. There is currently no C++ standard library supporting arbitrary time zones.

The C++ standard library already has the `<chrono>` library, which is a good implementation of an *absolute time* library. Another paper is proposing a standard *Civil-Time Library* ([P0215R0](#)). The current paper is proposing a standard *Time-Zone Library* that bridges `<chrono>` and the proposed Civil-Time Library, and completes the three pillars of the conceptual model just described.

Impact on the Standard

The Time-Zone Library proposed here depends on the existing `<chrono>` library with no changes. It also depends on the Civil-Time Library proposed in [P0215R0](#). This library is implementable using only C++98, and requires no language extensions.

Design Decisions

Use separately distributed time-zone data

This proposal depends on externally provided data that describes the rules for each time zone. Commonly this is distributed as data files, one for each time zone, as part of the IANA Time-Zone Database (<https://www.iana.org/time-zones>). These data may alternatively be located elsewhere on a computer (e.g., in a registry). The data source for the time zone library is implementation defined.

Leap seconds are disregarded (though could be supported)

Like most places, Google [disregards leap seconds](#), and therefore the Time-Zone Library presented here will also disregard them. However, if leap second support becomes necessary, it could be added to this library with minimal modification to the interface and some additional complexity for programmers.

Technical Specification

Time zones are canonically identified by a string of the form "*continent/city*", such as "America/New_York", "Europe/London", and "Australia/Sydney". The data encapsulated by a time zone describes the offset from the [UTC](#) time standard, a short abbreviation string (e.g. "EST" and "PDT"), and information about daylight-saving time (DST). These rules are defined by local governments and they may change over time. A time zone, therefore, represents the complete history of time-zone rules and when each rule applies for a given region.

Conceptually, a time zone represents the rules necessary to convert any *absolute time* to a *civil time* and vice versa.

[UTC](#) itself is naturally represented as a time zone having a constant zero offset, no DST, and an abbreviation string of "UTC". Treating UTC like any other time zone enables programmers to write correct, time-zone-agnostic code without needing to special-case UTC.

The core of the Time-Zone Library presented here is a single class named `time_zone`, which enables converting between absolute time and civil time. Absolute times are represented by `std::chrono::time_point` (on the `system_clock`), and civil times are represented using `civil_second` as described in the proposed Civil-Time Library ([P0215R0](#)). The Time-Zone Library also defines functions to format and parse absolute times as strings.

Synopsis

The interface for the core `time_zone` class is as follows.

```
#include <chrono>
#include "civil.h" // See proposal [P0215R0]

// Convenience aliases. Not intended as public API points.
template <typename D>
using time_point = std::chrono::time_point<std::chrono::system_clock, D>;
using sys_seconds = std::chrono::duration<std::chrono::system_clock::rep,
std::chrono::seconds::period>;

class time_zone {
public:
    time_zone() = default; // Equivalent to UTC
    time_zone(const time_zone&) = default;
    time_zone& operator=(const time_zone&) = default;

    struct absolute_lookup {
        civil_second cs;
        int offset; // seconds east of UTC
        bool is_dst; // is offset non-standard?
        std::string abbr; // time-zone abbreviation (e.g., "PST")
    };
    template <typename D>
    absolute_lookup lookup(const time_point<D>& tp) const;

    struct civil_lookup {
        enum civil_kind {
            UNIQUE, // the civil time was singular (pre == trans == post)
            SKIPPED, // the civil time did not exist
            REPEATED, // the civil time was ambiguous
        } kind;
        time_point<sys_seconds> pre; // Uses the pre-transition offset
        time_point<sys_seconds> trans; // Instant of civil-offset change
        time_point<sys_seconds> post; // Uses the post-transition offset
    };
    civil_lookup lookup(const civil_second& cs) const;

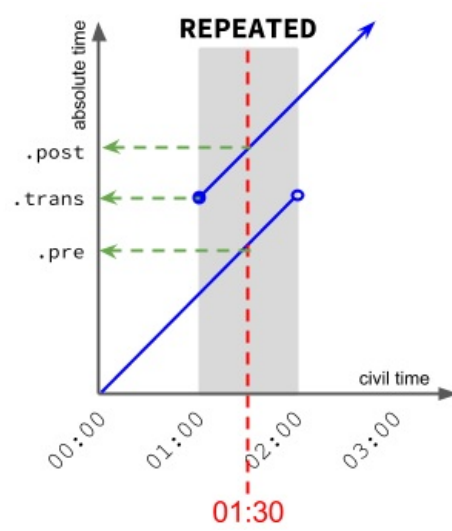
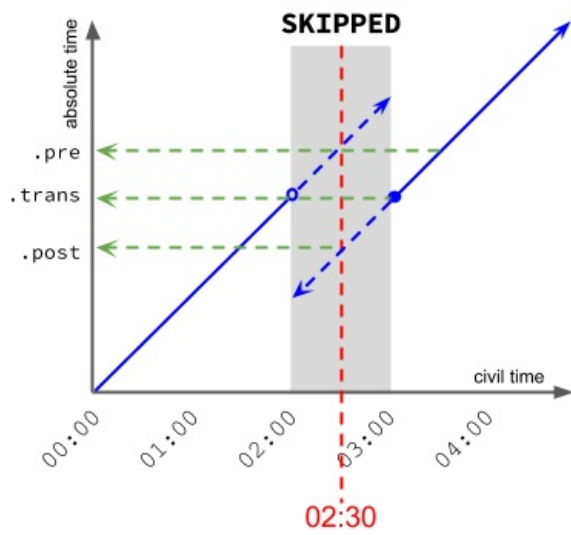
private:
    // ...
};

// Loads the named time zone. Returns false on error.
bool load_time_zone(const std::string& name, time_zone* tz);

// Returns a time_zone representing UTC. Cannot fail.
time_zone utc_time_zone();

// Returns a time zone representing the local time zone.
// Falls back to UTC.
time_zone local_time_zone();
```

Converting from an absolute time to a civil time (e.g., `std::chrono::time_point` to `civil_second`) is an exact calculation with no possible time-zone ambiguities. However, conversion from civil time to absolute time may not be exact. Conversions around UTC offset transitions may be given ambiguous civil times (e.g., the 1:00 am hour is repeated during the Autumn DST transition in the United States), and some civil times may not exist in a particular time zone (e.g., the 2:00 am hour is skipped during the Spring DST transition in the United States). The `time_zone::civil_lookup` struct gives callers all relevant information about the conversion operation, as the following diagram illustrates.



The full information provided by the `time_zone::absolute_lookup` and `time_zone::civil_lookup` structs is frequently not needed by callers. To simplify the common case of converting between `std::chrono::time_point` and `civil_second`, the Time-Zone Library provides an overloaded non-member `convert()` function that directly converts from one type to the other. These overloads are the main interface points that callers should use when converting between the absolute-time and civil-time domains, because they intelligently select a good default when time-zone uncertainties arise.

The implementation of these `convert()` functions must select an appropriate time point to return in cases of ambiguous/skipped civil-time conversions. The value chosen is such that the relative ordering of civil times is preserved when they are converted to absolute times. That is, given `civil_second a, b`; if $a < b$, then `convert(a, tz) <= convert(b, tz)`.

```
template <typename D>
inline civil_second convert(const time_point<D>& tp, const time_zone& tz) {
    return tz.lookup(tp).cs;
}

inline time_point<sys_seconds> convert(const civil_second& cs, const time_zone& tz) {
    const time_zone::civil_lookup lookup = tz.lookup(cs);
    if (lookup.kind == time_zone::civil_lookup::SKIPPED)
        return lookup.trans;
    return lookup.pre;
}
```

Finally, the Time-Zone Library provides functions for formatting and parsing absolute times with respect to a given time zone. These functions use `strftime()`-like format specifiers, with the following extensions:

Specifier	Description
%Ez	RFC3339-compatible numeric time-zone offset (+hh:mm or -hh:mm)
%E#S	Seconds with # digits of fractional precision
%E*S	Seconds with full fractional precision (a literal '*') %E4Y Four-character years (-999 ... -001, 0000, 0001 ... 9999)

```

template <typename D>
std::string format(const std::string& format, const time_point<D>& tp,
                  const time_zone& tz);
// Uses a format string of "%Y-%m-%dT%H:%M:%E*S%Ez"
template <typename D>
std::string format(const time_point<D>& tp, const time_zone& tz);

template <typename D>
bool parse(const std::string& format, const std::string& input,
          const time_zone& tz, time_point<D>* tpp);

```

Examples

Creating a time_zone

Time zones are created by passing the time zone's name to the `load_time_zone()` function along with a pointer to a `time_zone`. The function will return `false` if the named zone fails to load.

Additionally, callers may get time zones representing UTC, or the process's local time zone, through convenience functions that cannot fail and return the time zone by value.

```

time_zone nyc;
if (!load_time_zone("America/New_York", &nyc)) { /* error */ }

const time_zone utc = utc_time_zone();
const time_zone local = local_time_zone();

```

Creating a time_point from a civil_second

Converting from the civil-time domain to the absolute-time domain is one of the two fundamental operations of a time zone.

```

const time_zone utc = utc_time_zone();
const civil_second cs(2015, 2, 3, 4, 5, 6); // 2015-02-03 04:05:06

const auto tp1 = convert(cs, utc); // Civil -> Absolute

time_zone nyc;
if (!load_time_zone("America/New_York", &nyc)) { /* error */ }

const auto tp2 = convert(cs, nyc); // Civil -> Absolute
// tp1 != tp2

```

Creating a civil_second from a time_point

Converting from the absolute-time domain to the civil-time domain is one of the two fundamental operations of a time zone.

```

const time_zone utc = utc_time_zone();
const time_t tt = 1234567890;
const auto tp = std::chrono::system_clock::from_time_t(tt);

const civil_second cs1 = convert(tp, utc); // Absolute -> Civil

time_zone nyc;
if (!load_time_zone("America/New_York", &nyc)) { /* error */ }

const civil_second cs2 = convert(tp, nyc); // Absolute -> Civil
// cs1 != cs2

```

Handling daylight-saving time

As mentioned above, converting from an absolute time to a civil time is never affected by time-zone complexities like DST. On the other hand, conversions going in the other direction could be specified as either skipped or repeated civil times. The `convert()` function used thus far will always work, either returning the exact answer or a

good alternative if no exact answer exists. Most users will simply want to use `convert()`. However, if a programmer would like to handle possibly inexact conversions explicitly, they may do so by calling the `time_zone::lookup()` member functions directly as the following examples show. (Note: It may help to consult [this diagram](#) while reading these examples.)

The following example considers 2015-03-08 02:30:00, which did not exist in New York, USA. This example illustrates a civil time that is "skipped" when the civil-time offset changes by +1 hours from UTC-0500 to UTC-0400.

```
const civil_second cs(2015, 3, 8, 2, 30, 0); // 2015-03-08 02:30:00
time_zone nyc;
if (!load_time_zone("America/New_York", &nyc)) { /* error */ }

const auto tp = convert(cs, nyc); // tp == 2015-03-08 03:00:00 -0400

const time_zone::civil_lookup lookup = nyc.lookup(cs);
// lookup.kind == time_zone::civil_lookup::SKIPPED
// lookup.pre == 2015-03-08 03:30:00 -0400 (== 2015-03-08 02:30:00 -0500)
// lookup.trans == 2015-03-08 03:00:00 -0400 (== 2015-03-08 02:00:00 -0500)
// lookup.post == 2015-03-08 01:30:00 -0500 (== 2015-03-08 02:30:00 -0400)
```

The next example considers 2015-11-01 01:30:00, which was repeated in New York, USA. This example illustrates a civil time that is "repeated" when the civil-time offset changes by -1 hours from UTC-0400 to UTC-0500.

```
const civil_second cs(2015, 11, 1, 1, 30, 0); // 2015-11-01 01:30:00
time_zone nyc;
if (!load_time_zone("America/New_York", &nyc)) { /* error */ }

const auto tp = convert(cs, nyc); // tp == 2015-11-01 01:30:00 -0400

const time_zone::civil_lookup lookup = nyc.lookup(cs);
// lookup.kind == time_zone::civil_lookup::REPEATED
// lookup.pre == 2015-11-01 01:30:00 -0400
// lookup.trans == 2015-11-01 01:00:00 -0500 (== 2015-11-01 02:00:00 -0400)
// lookup.post == 2015-11-01 01:30:00 -0500
```

Flight example

This [good example](#) is borrowed from Howard Hinnant.

There's nothing like a real-world example to help demonstrate things. Imagine a plane flying from New York, New York, USA to Tehran, Iran. To make it more realistic, let's say this flight occurred before the hostage crisis, right at the end of 1978. Flight time for a non-stop one way trip is 14 hours and 44 minutes.

Given that the departure is one minute past noon on Dec. 30, 1978, local time, what is the local arrival time?

```
time_zone nyc;
if (!load_time_zone("America/New_York", &nyc)) { /* error */ }
const auto departure = convert(civil_second(1978, 12, 30, 12, 1, 0), nyc);
const auto flight_length = std::chrono::hours(14) + std::chrono::minutes(44);
const auto arrival = departure + flight_length;
time_zone teh;
if (!load_time_zone("Asia/Tehran", &teh)) { /* error */ }
// format(departure, nyc) == 1978-12-30T12:01:00-05:00
// format(arrival, teh) == 1978-12-31T11:45:00+04:00
```

References

- https://en.wikipedia.org/wiki/Coordinated_Universal_Time
- https://en.wikipedia.org/wiki/Proleptic_Gregorian_calendar
- https://en.wikipedia.org/wiki/Tz_database
- <http://howardhinnant.github.io/tz.html>
- <https://github.com/google/cctz>